

3/8/23

* NAND - NAND - AND-OR SOP

* NOR - NOR - OR-AND POS

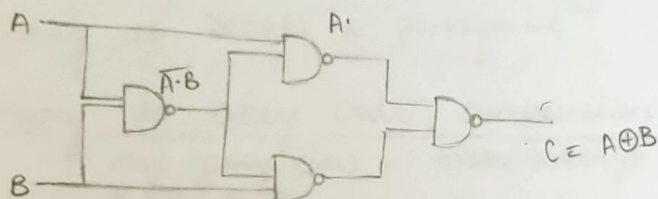
Implement :-

- i) EX-OR gate using minimum no. of NAND gate
- ii) EX-NOR using NOR gate
- iii) $Y = a \cdot (b+c)$ using NAND gate.

i) $C = A \oplus B$

$C = A \cdot \bar{B} + \bar{A} \cdot B$

$C =$



XOR Table

	a	b	Y
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

odd no. of 1 as input $\rightarrow \%_2 = 1$
even no. of 1 $\rightarrow \%_2 = 0$

$Y = \sum m(1, 2) = \bar{a}b + a\bar{b}$

$Y = \prod M(0, 3) = (a+b) \cdot (\bar{a} + \bar{b})$

SOP $\rightarrow$ Sum of products

minterm :- It is product term which contains all the variables either in complemented form or in uncomplemented form.

$Y = a + \bar{a}b$ - SOP

$= a(b + \bar{b}) + \bar{a}b$

$SSOP = \underbrace{ab + \bar{a}b + \bar{a}\bar{b}}_{\text{minterm}}$

Sum of minterms.

SPOS :- Standard POS $\rightarrow$ Product of max terms.

maxterm :- It is sum term which contains all variables either in complemented form or uncomplemented form.

$POS = \prod (\text{maxterms})$ $\%_2 \rightarrow 0, x = 0, \bar{x} = 1$

* Total combinations $= 2^n$
 $n \rightarrow$ no. of input

OR Gate

a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

$$y = a + b$$

$$y = \bar{a}b + b\bar{a} + ab$$

De-Morgan's $\rightarrow \overline{a+b} = \bar{a} \cdot \bar{b}$

$$y = \overline{\overline{\bar{a} \cdot b + a \cdot \bar{b}}}$$

$$= \overline{\bar{a} \cdot b \cdot \overline{a \cdot \bar{b}}}$$

3 1 2 4

So, Maxterm (XOR)

$$y = (a+b)(\bar{a}+\bar{b})$$

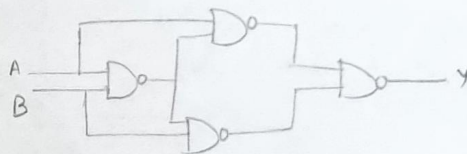
$$= (a+b) \cdot (\bar{a} \cdot \bar{b})$$

$$= a(\bar{a} \cdot \bar{b}) + b(\bar{a} \cdot \bar{b})$$

$$= \overline{a' + b'}$$

$$= \overline{a \cdot y' \cdot b \cdot y'}$$

$$y' = \bar{a} \cdot \bar{b}$$



Duality Theorem :- (-ve logic)

$$\bullet \longleftrightarrow +$$

Interchange / Swap

$$0 \longleftrightarrow 1$$

Interchange / Swap

Dual

$$\text{AND} \iff \text{OR}$$

$$A \cdot B$$

$$A + B$$

$$\text{NOT} \iff \text{NOT}$$

$$\bar{A}$$

$$\bar{A}$$

$$\text{NAND} \iff \text{NOR}$$

$$\overline{A \cdot B}$$

$$\overline{A + B}$$

$$\text{EX-OR} \iff \text{EX-NOR}$$

$$A\bar{B} + \bar{A}B$$

$$A \oplus B$$

$$(A + \bar{B}) \cdot (\bar{A} + B)$$

$$A\bar{A} + A\bar{B} + \bar{A}B + \bar{B}B$$

$$0 + A\bar{B} + \bar{A}B + 0$$

XNOR Gate

x	y	$x \oplus y$
0	0	1
0	1	0
1	0	0
1	1	1

$$y = \bar{a}\bar{b} + ab$$

$$y = (\bar{a} \cdot \bar{b}) + (a \cdot b)$$

→ If n no. of variables, combinations are :- 2^n
 no. of total logical expression = 2^{2^n}

ab	$a+b$	0	1
$\bar{a}b$	$\bar{a}+b$	a	b
$a\bar{b}$	$a+\bar{b}$	$\bar{a}$	$\bar{b}$
$\bar{a}\bar{b}$	$\bar{a}+\bar{b}$	$a \oplus b$	$a \odot b$

36

- 64 - 32 16 8 4 2 1

35 → 1 0 0 0 0 1 1

69 → 1 0 0 0 1 0 1

Binary to Octal & Hexagonal?

Binary to Grey code conversion :-

MSB (Grey code) = MSB (Binary)

$a_3 \oplus a_2 \oplus a_1 \oplus a_0$

↓
 G_3

$G_3 = a_3$

$G_2 = a_3 \oplus a_2$

$G_1 = a_2 \oplus a_1$

$G_0 = a_1 \oplus a_0$

$a + a' = 1$

Bin		Grey code
$a_3 a_2 a_1 a_0$		$a_3 a_2 a_1 a_0$
0000	→	0000
0001	→	0001
0010	→	0011
0011	→	0010
0100	→	0110
0101	→	0111
0110	→	0101
0111	→	0100
1000	→	1100
1001	→	1101
1010	→	1011
1011	→	1010
1100	→	1001
1101	→	1000
1110	→	1100
1111	→	1101

→ Grey code also known as unit distance code.

K-map :-

→ Graphical representation through which we minimize any boolean f^n .

→ K-map uses Grey code or unit distance code.

→ i) Total no. of cells/squares = 2^n n : no. of variables.

ii) Adjacent cells: From going one cell to another if only 2 variable changes. These 2 cells are called adjacent cells.

2 change

	BC	00	01	11	10
A	0	0	1	2	3
1		4	5	7	6

→ we make grouping of adjacent cells in 2^i form.
 & in this i variables are eliminated.

Ex:- if $i=3$, in an octate 3 variables will be eliminated.

Prime Implicants (PI)

→ It is a group of minterms or minterm.

Essential Prime Implicants (EPI)

→ It is prime implicant in which there is atleast one minterm which isn't in bond in any other group.

→ Final minimised expression will be sum of EPIs.

Ques: Minimize following Boolean f^n by using K-Map.
Also determine total no. of prime implicants & essential PIs.

$$Y = \sum m(1, 3, 6, 7)$$

		BC			
A	PI ₁ ← 0	00	01	10	11
			1	1	
1	1			1	1

PI₂ ↓

→ PI₃

$$PI_1 = EPI$$

$$PI_2 = \text{NOT EPI}$$

$$PI_3 = EPI$$

$$Y = \bar{A}C + AB$$

Consensus Theorem (Redundancy Theorem)

→

$$(\bar{a}+b) \cdot (b+c) \cdot (c+a)$$

$$= (\bar{a}+b) \cdot (c+a)$$

Reductant

$$\bar{a}b + bc + ca$$

$$\bar{a}b + ca$$

SOP form

POS form

← Quality

$$\bar{a}b + \bar{b}c + \bar{c}a = \bar{a}b + \bar{b}c + \bar{c}a$$

$$Y = \sum m(0, 1, 2, 3, 8, 9, 10)$$

AB	CD	00	01	11	10
		0	1	1	1
00		0	1	1	1
01		4	5	7	6
11		12	13	15	14
10		8	9	11	10

$$PI_1 = EPI$$

$$PI_2 = \text{NOT EPI}$$

$$PI_3 = EPI$$

3 Quads.

$$Y = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{B}\bar{D}$$

$$Y = \sum m(1, 5, 6, 7, 11, 12, 13, 15)$$

$$PI = 5, EPI = 4$$

AB \ CD	00	01	11	10
00		1		
01		1	1	1
11	1	1	1	
10			1	

$$Y = \bar{A}\bar{C}D + \bar{A}BC + ACD + ABC$$

$$Y = \pi M(0, 1, 2, 3, 4, 5, 8, 9, 10, 12, 13)$$

$$= \sum m(6, 7, 11, 14, 15)$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	0	1	1
10	0	0	1	0

$$Y = C \cdot (A+B)(B+D)$$

~~$$Y = ACB + ACD$$~~

$$Y = BC + ACD$$

$$= C(B+AD)$$

$$= C(B+A)(B+D)$$

To convert SOP $\Rightarrow$ POS : Apply distributive theorem.

$$x \cdot (y+z) = xy + xz$$

$$x + yz = (x+y)(x+z)$$

→

Boolean Algebra Theorem:

* Duality

Interchange $1 \Rightarrow 0$ and $+ \Rightarrow \cdot$.

$$1. x \cdot x = x$$

$$x + x = x$$

$$2. x \cdot 0 = 0$$

$$x + 1 = 1$$

$$3. (x')' = x$$

$$\overline{\overline{x}} = x$$

→ Involution

4. Associative & Distributive

$$* (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

$$(x+y)+z = x+(y+z)$$

$$* x \cdot (y+z) = (x \cdot y) + (x \cdot z)$$

$$x+(y \cdot z) = (x+y) \cdot (x+z)$$

5. De Morgan

$$(x \cdot y)' = x' + y'$$

$$(x+y)' = x' \cdot y'$$

$$\overline{x \cdot y} = \overline{x} + \overline{y}$$

$$\overline{(x+y)} = \overline{x} \cdot \overline{y}$$

6. Absorption

$$x \cdot (x + y) = x$$

$$x + (x \cdot y) = x$$

Q Minimize the following boolean f^n :

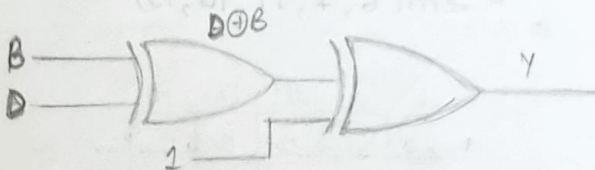
$$Y = \sum m(5, 8, 10, 13, 15) + \sum d(0, 1, 2, 3)$$

CD \ AB	00	01	11	10
00	X	X	X	X
01		1	1	
11		1	1	
10				1

$$Y = BD + \bar{B}\bar{C}\bar{D} + \bar{B}C\bar{D}$$

$$Y = BD + \bar{B}\bar{D}$$

$$BD(D + \bar{D}) + \bar{B}$$



XOR:-

$$a \oplus 1 = \bar{a} \text{ (NOT)}$$

$$a \oplus 0 = a \text{ (buffer)}$$

$$a \oplus a = 0$$

$$a \oplus \bar{a} = 1$$

XNOR:-

$$a \odot 0 = \bar{a}$$

$$a \odot 1 = a$$

$$a \odot a = 1$$

$$a \odot \bar{a} = 0$$

Q Minimize the following boolean f^n

CD \ AB	00	01	11	10
00	d	d	1	d
01		d	1	
11			1	d
10	d		1	1

$$Y = CD + AC$$

$$Y = CD + \bar{B}\bar{D}$$

$$Y = CD + \bar{B}C$$

$$Y = \bar{A}D + AC$$

CD \ AB	00	01	11	10
00	d	d		d
01	0	d		0
11	0	0		d
10	d	0		

$$Y = C \cdot (A + D) = AC + CD$$

$$Y = C \cdot (\bar{B} + D) = \bar{B}C + CD$$

$$Y = (\bar{B} + C) \cdot (\bar{B} + D)$$

$$Y = \bar{A}\bar{B} + \bar{A}D + \bar{B}C + CD$$

$$Y = (\bar{A} + C)(A + D) = \bar{A}D + AC + CD$$

$$= \bar{A}D + AC$$

[∴ consensus theorem]

$$Y = (C + \bar{D}) \cdot (D + \bar{B})$$

Q Minimize the following:-

AB \ CD	00	01	11	10
00	1	1	1	d
01	1	d	0	0
11	1	d	0	1
10	1	1	d	1

$$Y = \bar{C} + \bar{B} + \bar{A}D$$

5- Variable K-Map:-

$$\text{No. of cells} = 2^5 = 32$$

BC \ DE	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

BC \ DE	00	01	11	10
00	16	17	19	18
01	20	21	23	22
11	28	29	31	30
10	24	25	27	26

	A	B	C	D	E
A	0	0	0	0	0
A	0	1	1	1	1
A	1	0	0	0	0
A	1	1	1	1	1

Adjacent cells $\rightarrow a_{ij} \approx b_{ij}$

$$Y = \sum m(5, 7, 9, 11, 13, 15, 27, 29, 31, 21, 23)$$

$$Y = \bar{A}CE + \bar{A}BE + BCF + BDE$$

$$Y = CE + \bar{A}BE + BDE$$

BC \ DE	00	01	11	10
00	0	d ₁	d ₂	2
01	4	d ₅	7	6
11	d ₁₂	13	d ₁₅	14
10	d ₈	9	11	d ₁₀

BC \ DE	00	01	11	10
00	16	d	19	18
01	20	21	23	22
11	28	29	d	30
10	d	d	27	26

3 Octate
2 Quad

$$Y = \bar{B}\bar{C} + \bar{D}E + \bar{A}B + B\bar{C}\bar{E} + \bar{A}B$$

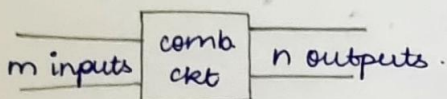
Q

BC \ DE					
BC	DE	00	01	11	10
	00	d	1	1	d
	01		d	1	
	11			1	
	10			1	

$$Y = DE + \bar{B}\bar{C}$$

BC \ DE					
BC	DE	00	01	11	10
	00	d	1	1	d
	01			d	
	11			1	d
	10	d	d	1	d

Design Procedure of Combinational Circuits:-



Step 1: Determine total no. of inputs and outputs.

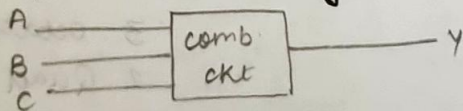
$$\begin{array}{l} a+b \\ a \text{ \% } b \end{array} \quad \begin{array}{l} \text{Sum} \\ \text{Sum carry} \end{array} \quad \begin{array}{l} \% \\ \text{Carry} \end{array}$$

Step 2: Draw the truth table with the help of given statement.

Step 3: With the help of boolean algebra or K-Map, minimize every output which is functions of inputs.

Step 4: With the help of suitable logic gates implement every output f^n .

Q A combinational circuit has 3 inputs and 1 output. Output becomes high when majority no. of inputs are high, otherwise low.

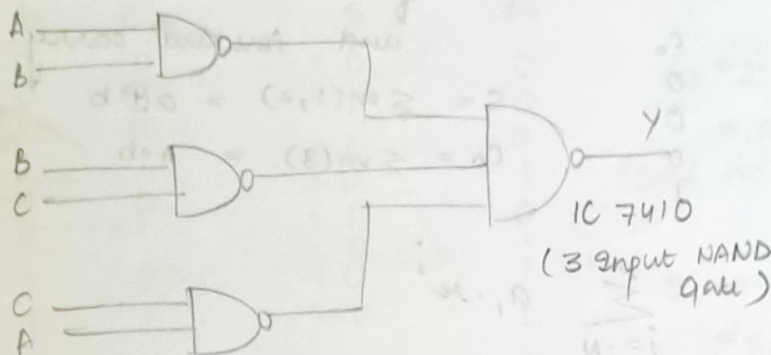
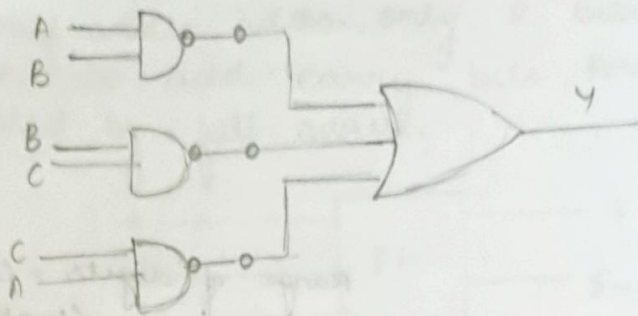


I/P			%P
A	B	C	
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$Y = \sum m(3, 5, 6, 7)$$

BC \ A					
A	BC	00	01	11	10
				1	
0				1	
1		1	1	1	

$$Y = ab + bc + ca$$



Q Design a combinational ckt which has 4 inputs and 2 output. Output becomes high when it is divisible by 5 otherwise low.

	A	B	C	D	%
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	1
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	1

→ Both bits are equal

	a	b	Y
0	0	0	1
1	0	1	0
2	1	0	0
3	1	1	1

XNOR

%

$$Y = \sum m(0, 5, 10, 15)$$

CD \ AB	00	01	11	10
00	1			
01		1		
11			1	
10				1

$$Y = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + A\bar{B}C\bar{D} + A\bar{B}C\bar{D}$$

$$Y = \bar{A}\bar{C}(\bar{B}\bar{D} + BD) + AC(\bar{B}\bar{D} + BD)$$

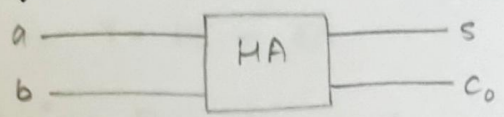
$$Y = \bar{A}\bar{C}(b \odot d) + AC(b \odot d)$$

$$Y = (b \odot d)(a \odot c)$$

XOR is also known as:-

1. Coincidence ckt
2. Equivalent detector
3. Inclusive OR Gate

Half Adder



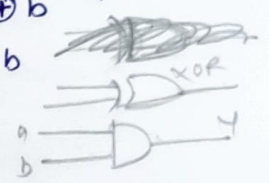
Range of digits = 0 to $b-1$
($b \rightarrow$ base)

* If sum $> b-1$, subtract b and forward carry.

a	b	s	c ₀
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$s = \sum m(1,2) = a \oplus b$$

$$c_0 = \sum m(3) = a \cdot b$$



$()_x \rightarrow ()_{10}$

$$\sum_{i=-N}^{N=1} a_i \cdot x^i$$

Q. $\frac{55}{5} = 11$

$$\frac{5 + 5x^1}{5x^0} = 1x^0 + 1x^1$$

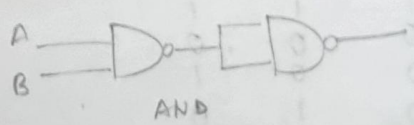
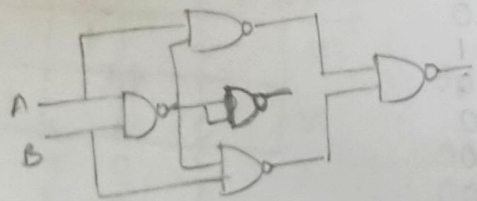
$$\frac{5 + 5x}{5} = 1 + x$$

$$5 + 5x = 5 + 5x$$

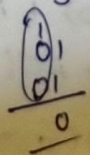
or = x ??

→ Here 5 isn't possible upto base 5.

Q. Implement half adder by using minimum no. of NAND gates.

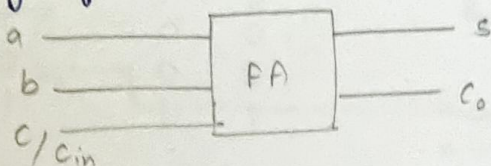


→ Drawback of HA
Can't add 3 bits (carry bit). Hence full adder is introduced (used)



Full Adder

→ Half adder adds only 2 bits & there is no provision to add carry bits. And this problem is avoided by full adder.



a	b	Cin	S	C ₀
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$C_0 = \sum m(3, 5, 6, 7)$$

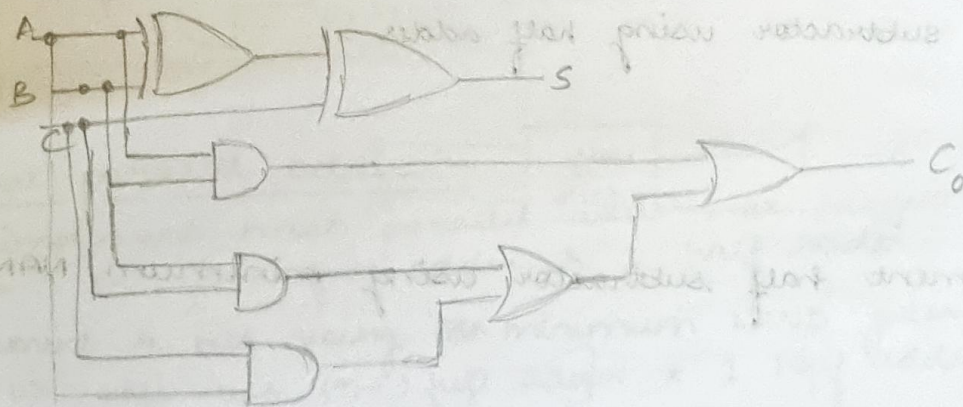
$$= ab + bc_{in} + c_{in}a$$

(Odd no. of inputs are high, output high)

$$S = a \oplus b \oplus C_{in}$$

$$= \sum m(1, 2, 4, 7)$$

* XOR is only 2 input.



* C_0 = when majority no. of inputs are high.

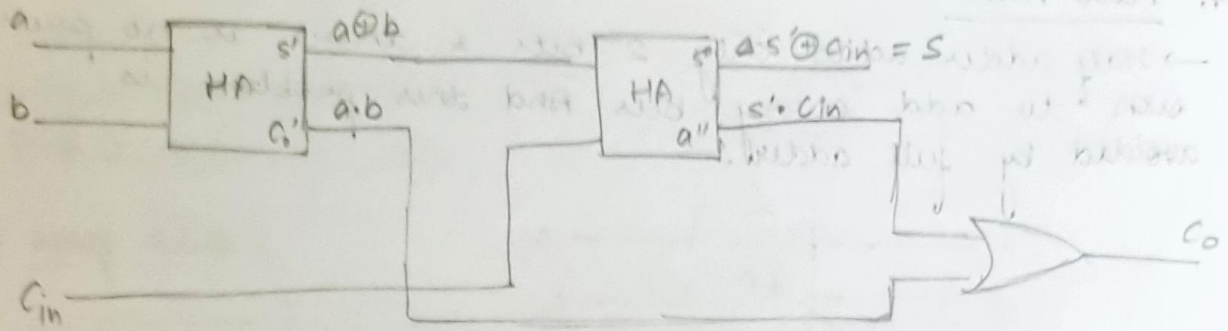
* S = odd no. of inputs are high.

Q Implement full adder by using half adder & suitable logic gate.

$$\rightarrow C_0 = \sum m(3, 5, 6, 7)$$

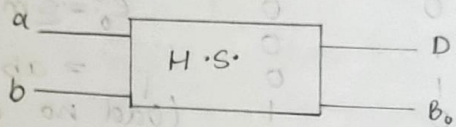
$$= \bar{a}bc_{in} + a\bar{b}c_{in} + ab\bar{c}_{in} + abc_{in}$$

$$= (a \oplus b) \cdot c_{in} + a \cdot b$$



17/8/23

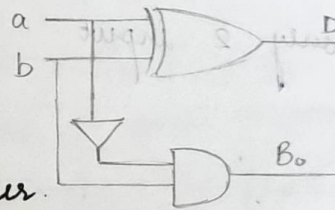
Half Subtractor



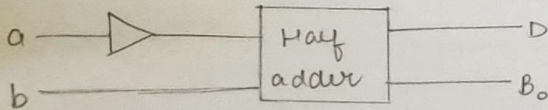
a	b	D	B _o
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$D = a \oplus b$$

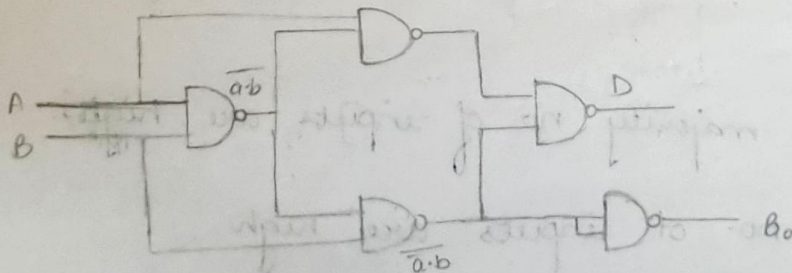
$$B_o = \bar{a}b$$



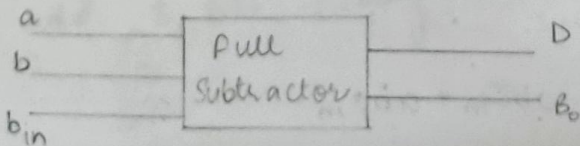
Half subtractor using half adder.



Q Implement half subtractor using minimum NAND gates.



Full Subtractor



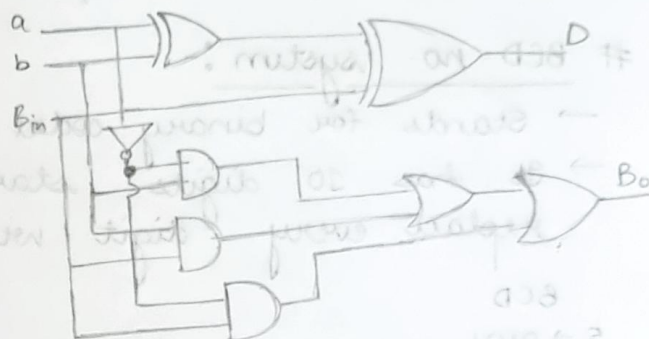
	a	b	b _{in}	D	B ₀
0	0	0	0	0	0
1	0	0	1	1	1
2	0	1	0	1	1
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	0
6	1	1	0	0	0
7	1	1	1	1	1

$$D = \sum m(1, 2, 4, 7)$$

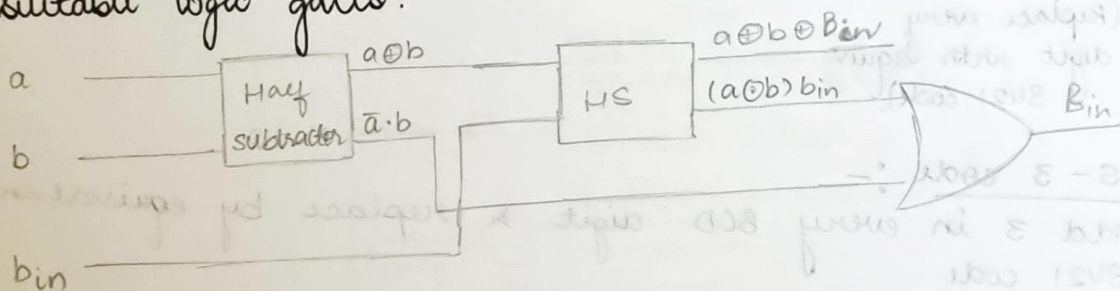
$$D = a \oplus b \oplus B_{in}$$

$$B_0 = \sum m(1, 2, 3, 7)$$

$$B_0 = \bar{a}b + bB_{in} + B_{in}\bar{a}$$



Q Implement full subtractor using half subtractor & suitable logic gates.



n-bit parallel adder:-

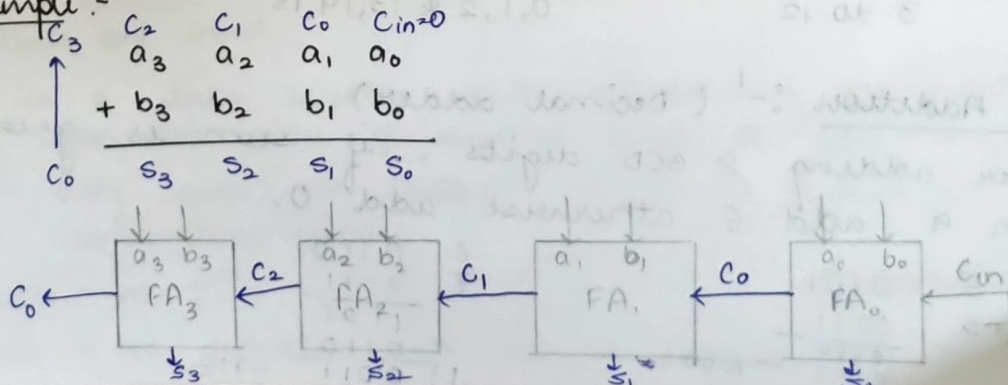
→ To implement n-bit parallel adder, we require n full adders or n-1 full adder & 1 half adder.

* Implement 4 bits using minimum NAND gates.

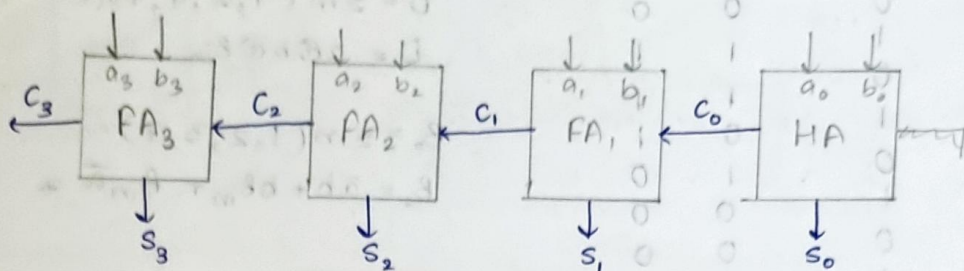
→ we will use (n-1) full adder & 1 half adder.

→ No. of NAND gates = (n-1)*9 + 1*5

Example:-



using half adder & full adder.



BCD no. system :-

- Stands for binary coded decimal.
- It has 10 digits starting from 0 to 9 & replace every digit with equivalent 8421 code (4 bit code).

BCD

5 → 0101

8 → 1000

13 → 0001 0011

(Replace every digit with equiv. 8421 code)

XS-3 code :-

- Add 3 in every BCD digit & replace by equivalent 8421 code.

Q Determine XS-3 code of :-

1. 5 → 1000

2. 8 → ~~0000~~ 1011

3. 15 → 001001000

$\frac{33}{48}$

4. $(100011)_2 \rightarrow 00101000$

5. $(37)_8 \rightarrow 01100100$

$\frac{35}{22}$

BCD	Valid
	0 to 9
XS-3	3 to 12

Invalid (don't care)

10 to 15

0, 1, 2 & 13, 14, 15

29/08/23

BCD Addition :- (Decimal adder)

- After adding 2 BCD digits, if sum is greater than 9, add 6 otherwise add 0.

$$\begin{array}{r} 5 \quad 0101 \\ +3 \quad 0011 \\ \hline 1000 \rightarrow 1000 \end{array}$$

$$\begin{array}{r} 6 \quad 0110 \\ 7 \quad 0111 \\ \hline 1101 \\ +6 \quad 0110 \\ \hline 110011 \rightarrow 13 \end{array}$$

$$\begin{array}{r} 3 \quad 0011 \\ + 5 \quad 0101 \\ \hline 1000 \\ 0000 \\ \hline 1000 \\ s_3 s_2 s_1 s_0 \end{array}$$

$$\begin{array}{r} 5 \quad 0101 \\ + 6 \quad 0110 \\ \hline 1011 > 9 \\ 0110 + 6 \\ \hline 10001 \end{array}$$

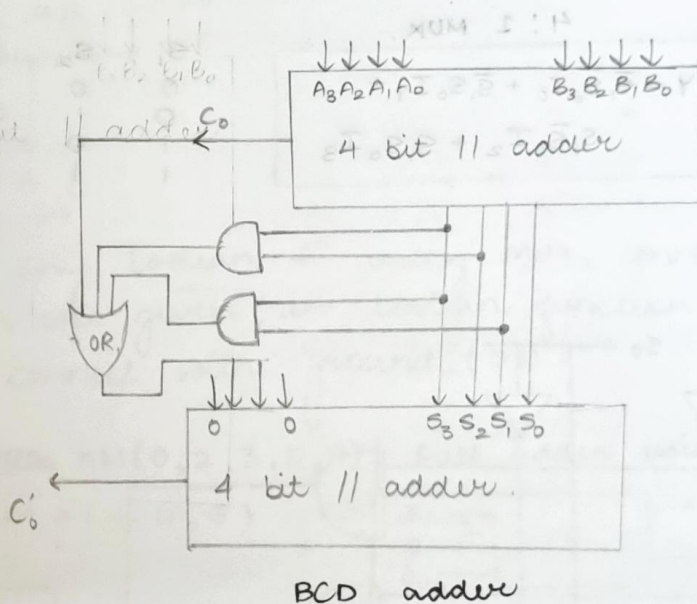
$$\begin{array}{r} 8 \quad 1000 \\ + 9 \quad 1001 \\ \hline 1) 0001 \\ 0110 \\ \hline 1) 0111 \\ 17 \checkmark \end{array}$$

* Requires 2 parallel adder.

If result > 9, add 6.

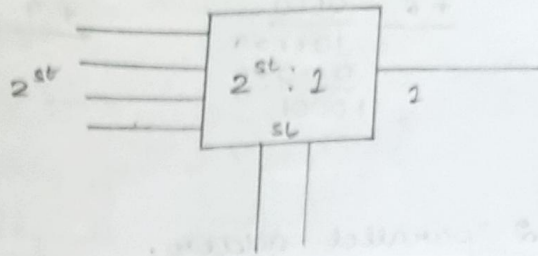
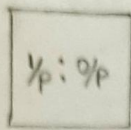
$s_3 s_2$ \ $s_1 s_0$	00	01	11	10
00				
01				
11	1	1	1	1
10			1	1

Imp:



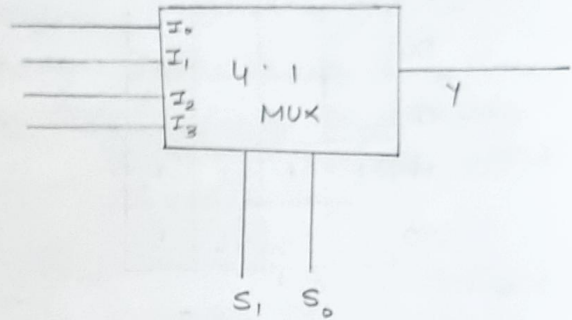
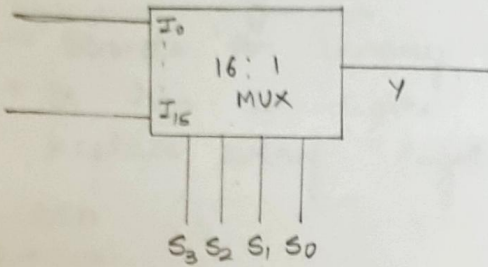
Multiplexer :- (MUX)

- data selector
- selects one input
- It is a data selector which selects only one input depending upon the value of selected lines. It has many inputs but only gives one output. so it is known as many to one combination circuit.
- Inputs = $m = 2^{\text{st}}$ selected lines (st)
- No. of st depends upon no. of inputs.



16:1 MUX

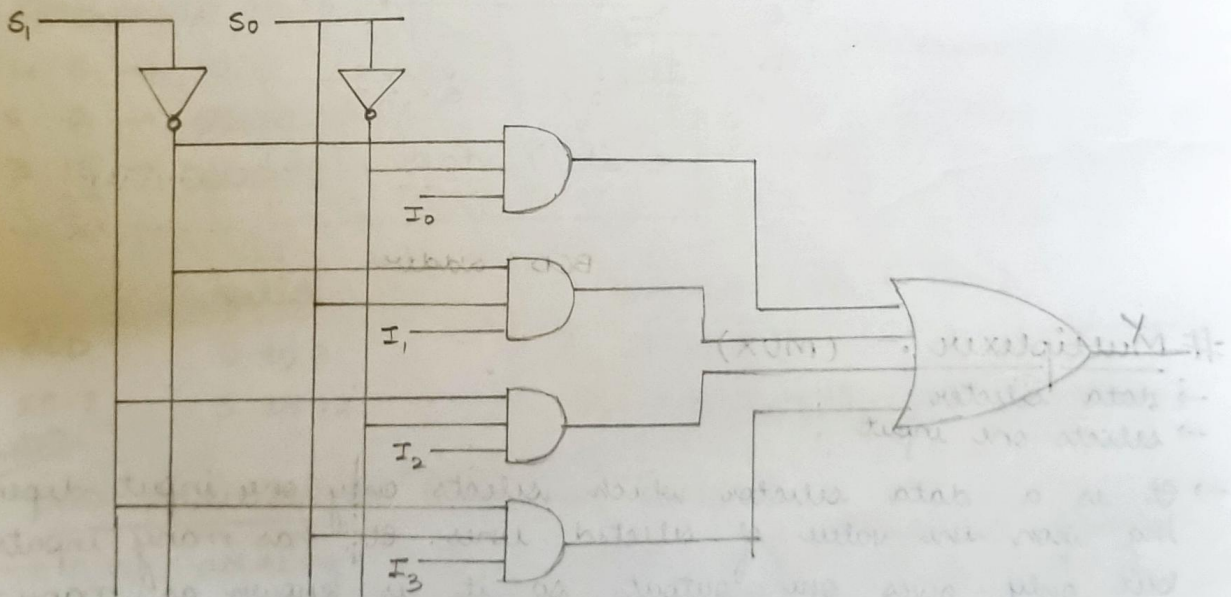
4:1 MUX



4:1 MUX.

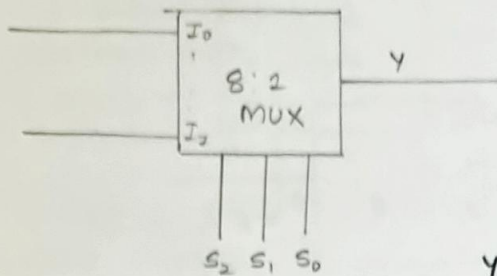
$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



→ In multiplexer we get output in SOP form.

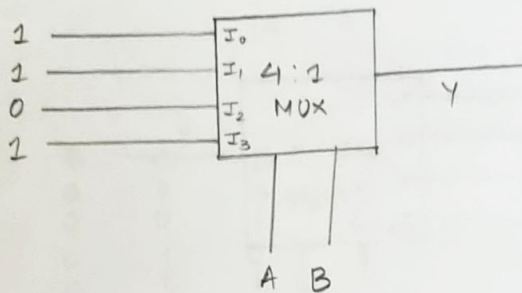
8:1 MUX



S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

$$Y = \bar{S}_2 \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_2 \bar{S}_1 S_0 I_1 + \bar{S}_2 S_1 \bar{S}_0 I_2 + \bar{S}_2 S_1 S_0 I_3 + S_2 \bar{S}_1 \bar{S}_0 I_4 + S_2 \bar{S}_1 S_0 I_5 + S_2 S_1 \bar{S}_0 I_6 + S_2 S_1 S_0 I_7$$

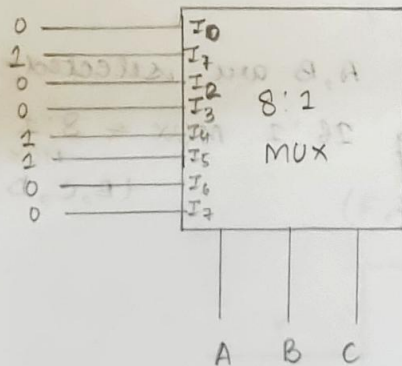
Q Implement $Y = \sum m(0, 1, 3)$ using 4:1 MUX.



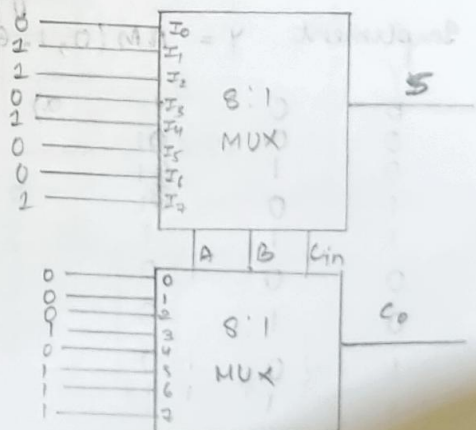
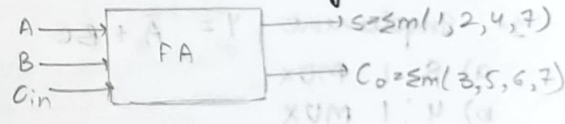
* To implement any boolean f^n using MUX, connect all the minterms which are given in boolean function with +5 (logic) volt otherwise connect with ground (0V).

Q Implement $Y = \sum m(0, 2, 3, 6, 7)$

$$Y = \sum m(1, 4, 5)$$



Full adder using 8:1 MUX.

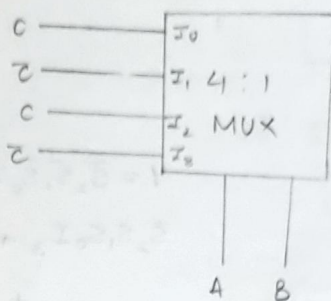


Q Implement $Y = \sum m(1, 2, 5, 6)$ using 4:1 MUX. (A, B, C)
 $= \bar{B}C + B\bar{C}$



→ Truth Table

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

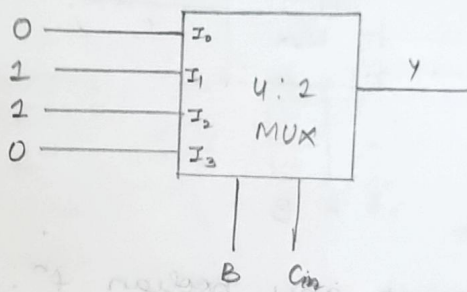


$$\begin{aligned}
 Y &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + AB\bar{C} \\
 &= \bar{B}C(\bar{A}+A) + B\bar{C}(\bar{A}+A) \\
 &= \bar{B}C + B\bar{C}
 \end{aligned}$$

a) When A, B are selected lines.

b) When B, C are selected lines.

A	B	C	Y
0	0	0	0
1	0	0	0
0	0	1	1
1	0	1	1
0	1	0	1
1	1	0	1
0	1	1	0
1	1	1	0



c) When A, C are selected line.

1. Implement $Y = \bar{A} + B\bar{C}$ using

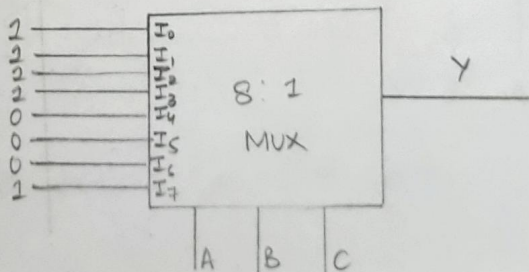
- 8:1 MUX
- 4:1 MUX

2. Full subtractor using 4:1 MUX. when A, B are selected.

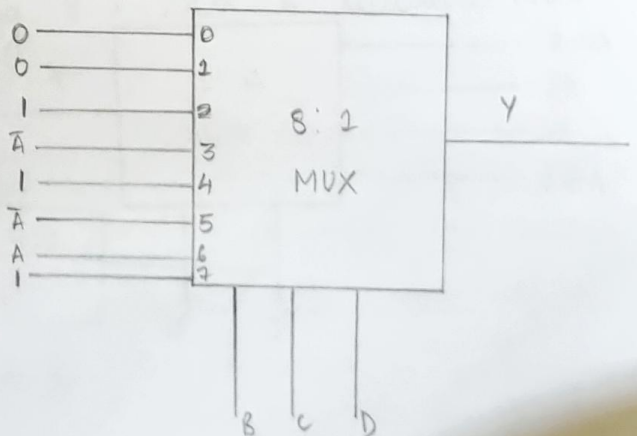
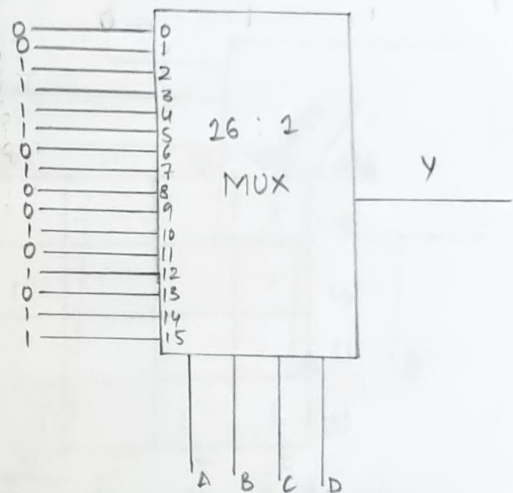
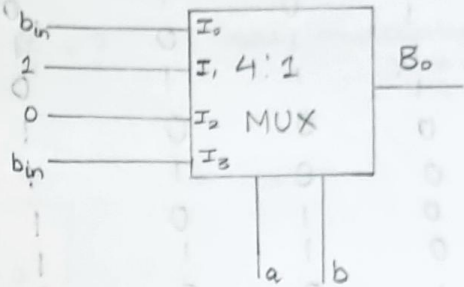
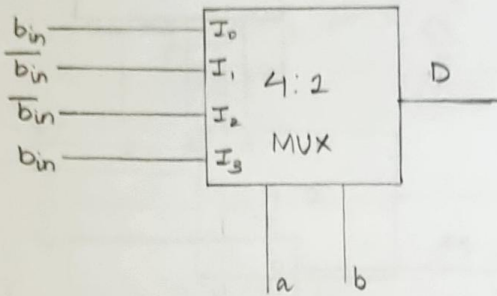
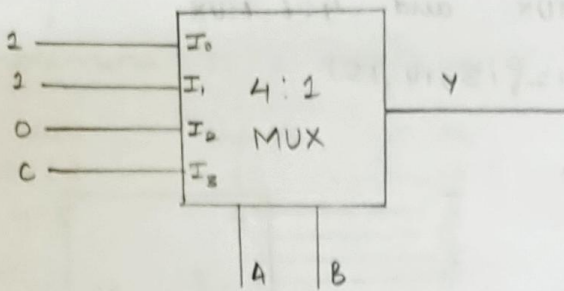
3. Implement $Y = \sum m(0, 1, 6, 8, 9, 11, 13)$ using 16:1 MUX & 8:1 MUX (B, C, D)

a) $Y = \sum m(0, 1, 2, 3, 7)$

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



b)



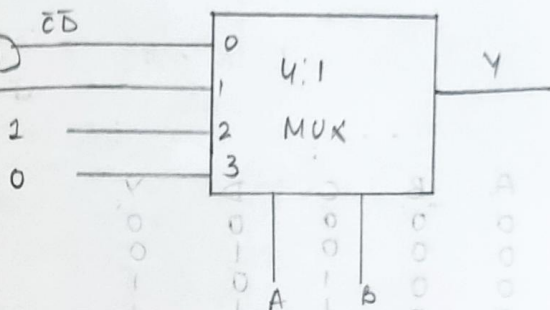
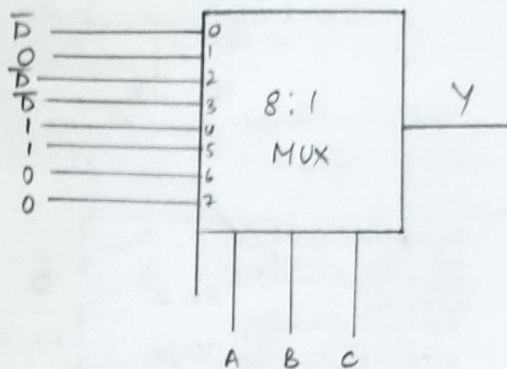
3.

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Q Implement using 8:1 MUX and 4:1 MUX.

$$Y = \sum m(1, 2, 3, 5, 7, 12, 13, 14, 15)$$

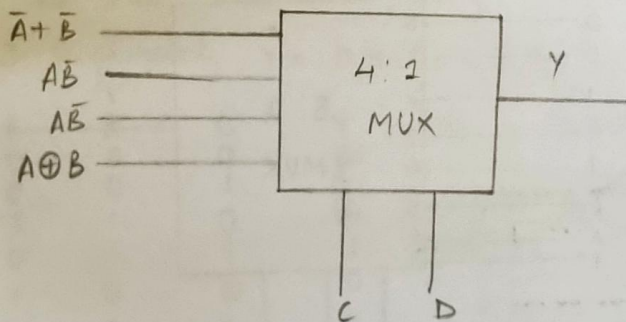
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0



CD \ AB	00	01	11	10
00	1			
01	1			1
11				
10	1	1	1	1

$I_0 = \bar{C} \cdot \bar{D}$
 $I_1 = \bar{D}$
 $I_2 = 0$
 $I_3 = 1$

Q If CD are selected lines.



Multiplexer Tree :-

Q Implement 26:1 MUX using 4:1 MUX.

MUX	Inputs	SL	O/P
26:1	26	4	1
4:1	4	2	1

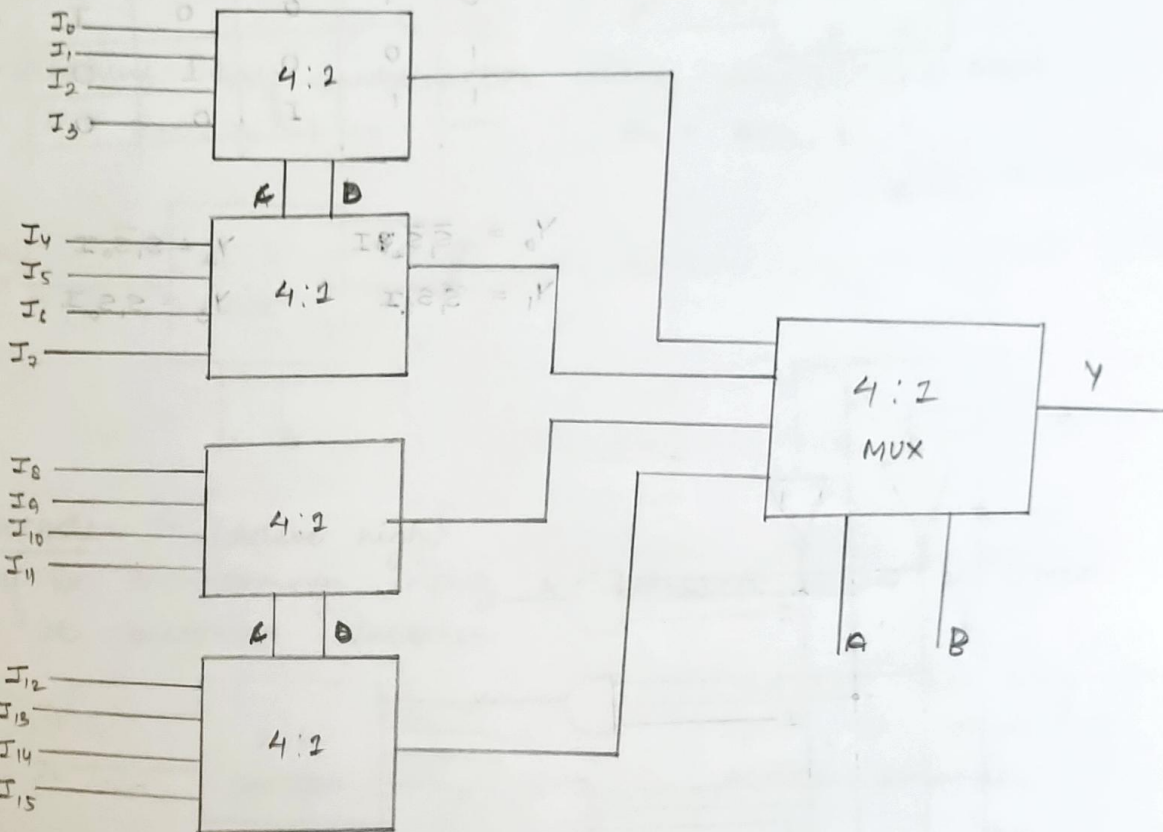
$$\frac{26}{4} = \frac{4}{4} = 1$$

$$\text{Total multiplexers} = 4 + 1 = 5$$

* 64:1 using 4:1

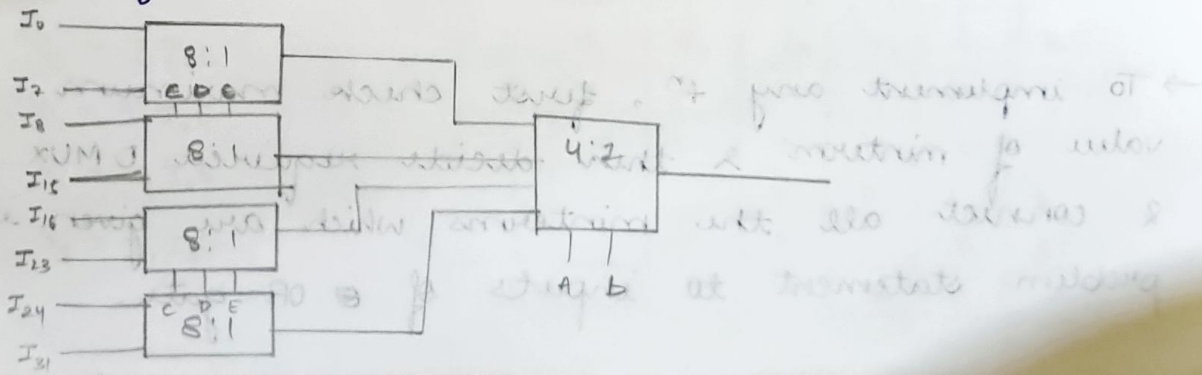
$$\frac{64}{4} = \frac{16}{4} = \frac{4}{4} = 1$$

$$\text{Total multiplexers} = 21$$



Q Implement 32:1 MUX using 8:1 MUX & suitable MUX.

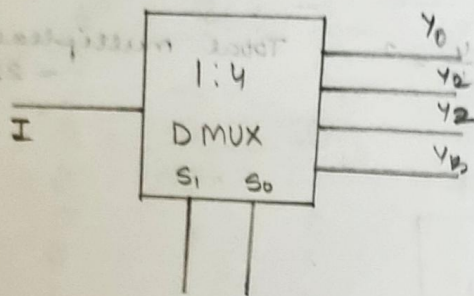
$$\frac{32}{8} = 4$$



Demultiplexer :-

- It is the reverse process of multiplexer.
- It has one inputs & many outputs.
- Also known as data distributor.

Total outputs : 2^{st} st : selected lines



S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

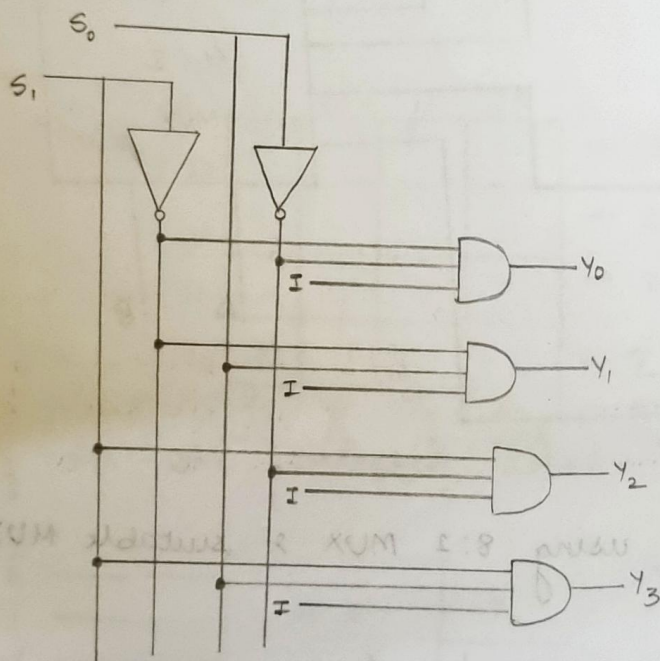
$$Y_0 = \bar{S}_1 \bar{S}_0 I$$

$$Y_1 = \bar{S}_1 S_0 I$$

$$Y_2 = S_1 \bar{S}_0 I$$

$$Y_3 = S_1 S_0 I$$

AND
terms

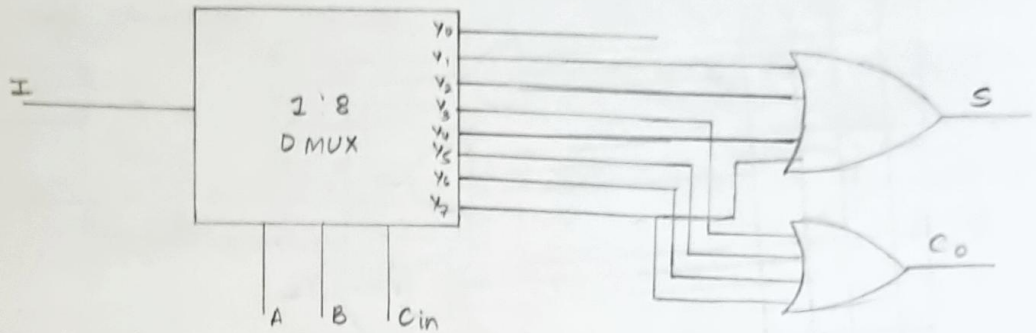


→ To implement any f^n , first check maximum value of minterm & then decide required DMUX & connect all the minterms which are given in problem statement to inputs of OR gate.

Q Implement full adder using suitable D MUX.

$$\rightarrow S = \sum m(1, 2, 4, 7)$$

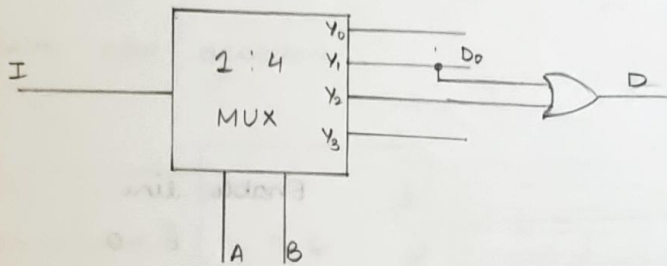
$$C_0 = \sum m(3, 5, 6, 7)$$



Q Implement half subtractor using suitable D-MUX.

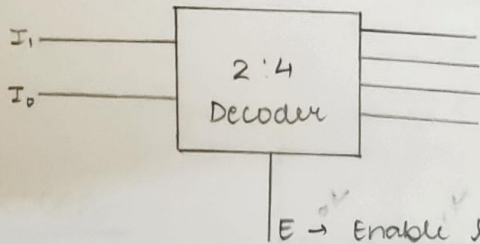
$$D = \sum m(1, 2)$$

$$B_0 = \sum m_1$$



Decoder :- (Active high)

→ If we interchange input & selected lines of D MUX, it becomes decoder.



* Decoder will only work when the value of enable line is 1.

Total outputs: $2^{i/p}$

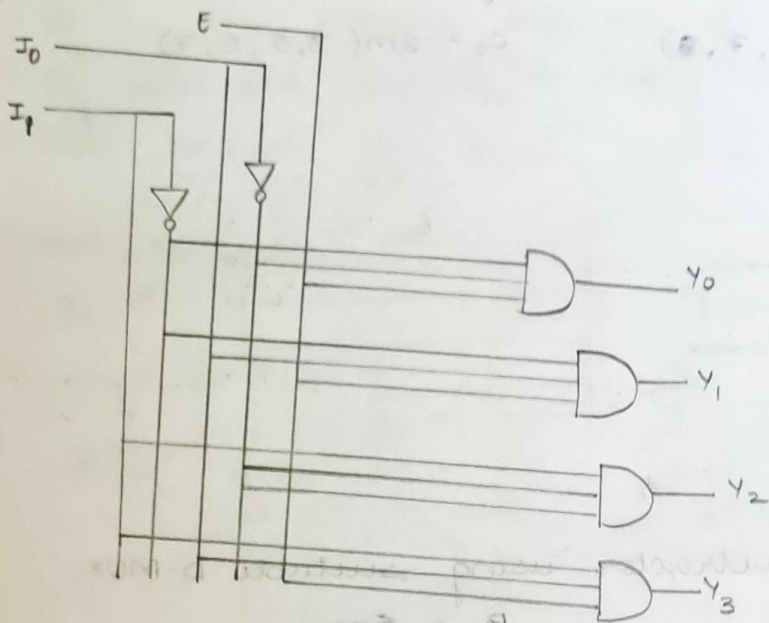
$$y_0 = \bar{I}_1, \bar{I}_0, E$$

$$y_1 = \bar{I}_1, I_0, E$$

$$y_2 = I_1, \bar{I}_0, E$$

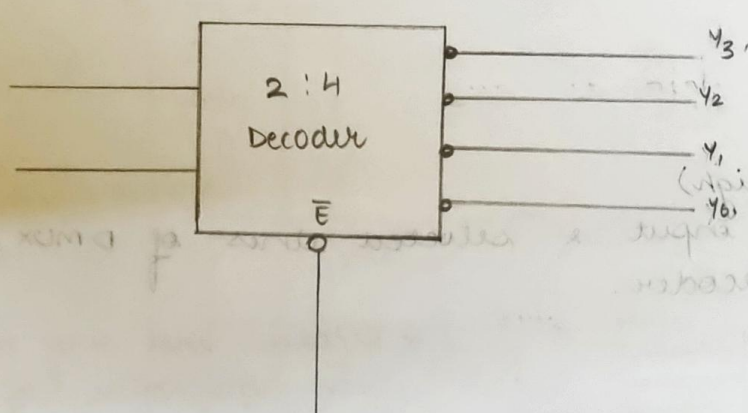
$$y_3 = I_1, I_0, E$$

E	I ₁	I ₀	y ₃	y ₂	y ₁	y ₀
0	X	X	Invalid			
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



Active-high

Active low decoder -



Enable line

$\bar{E} = 0$

→ works only if $\bar{E} = 0$

$\bar{E}$	I_1	I_0	Y_3	Y_2	Y_1	Y_0
1	x	x	invalid			
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1

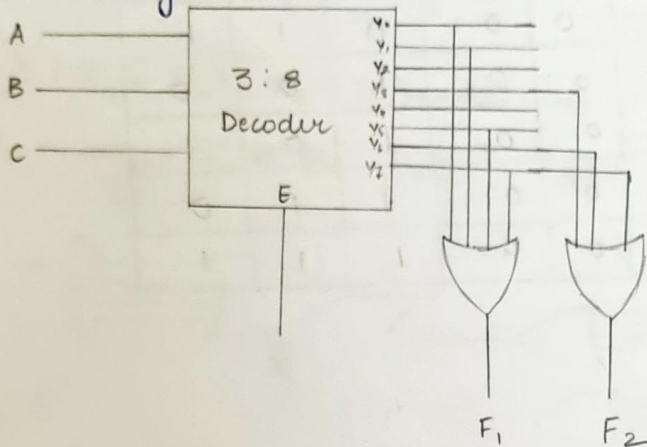
Q Implement following functions using

- Active high decoder
- Active low decoder

$$F_1(A, B, C) = \sum m(0, 1, 5, 7)$$

$$F_2(A, B, C) = \prod M(0, 1, 2, 4, 5) = \sum m(3, 6, 7)$$

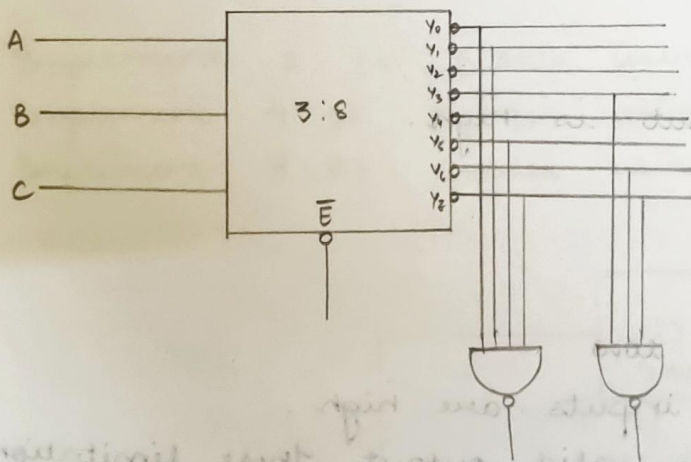
→ Active high decoder



$$F_2 = Y_3 + Y_6 + Y_7$$

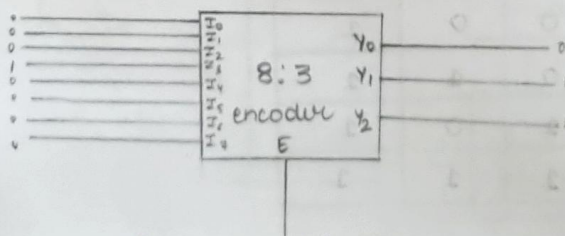
$$= \overline{Y_3} \cdot \overline{Y_6} \cdot \overline{Y_7}$$

Active low decoder



ENCODER

- Reverse process of decoder.
- Inputs 2^n and n outputs.

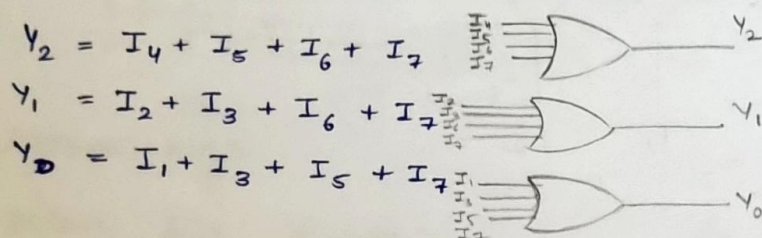


* octal to binary encoder.

→ converts octal to binary.

E = 1.

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1



Limitations:-

- All inputs are 0.
- More than one input is high.

Priority Encoder:-

• In normal encoder:

- if all inputs are low
- if more than one inputs are high.

we do not get any valid output. These limitations are avoided by Priority Encoder.

I_3	I_2	I_1	I_0	Y_1	Y_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

$$Y_1 = \pi M(1, 2, 3) + \sum d(0)$$

$$Y_0 = \pi M(1, 4, 5, 6, 7) + \sum d(0)$$

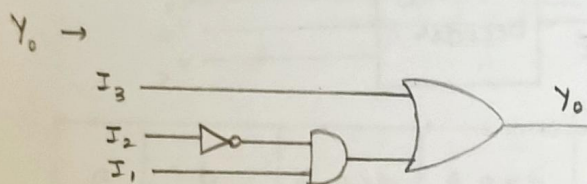
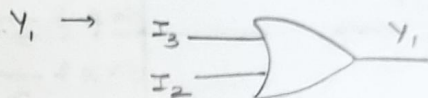
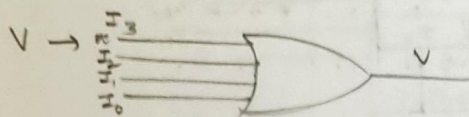
$$V = M0 = I_3 + I_2 + I_1 + I_0$$

$$Y_1 = (I_3 + I_2) = \overline{\overline{I_3} \cdot \overline{I_2}}$$

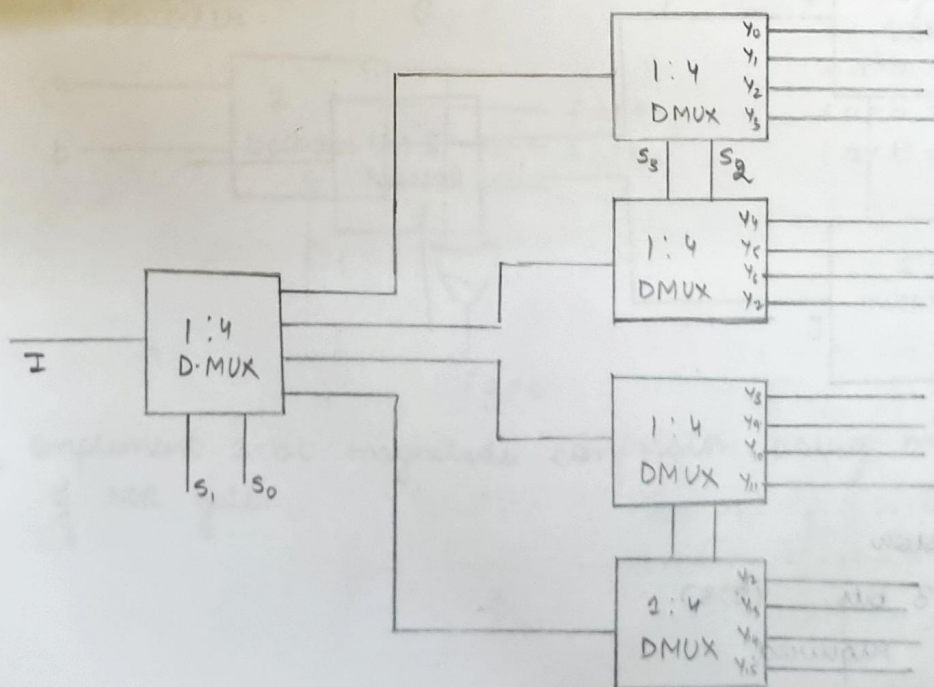
$$Y_0 = (I_3 + \overline{I_2})(I_3 + I_1)$$

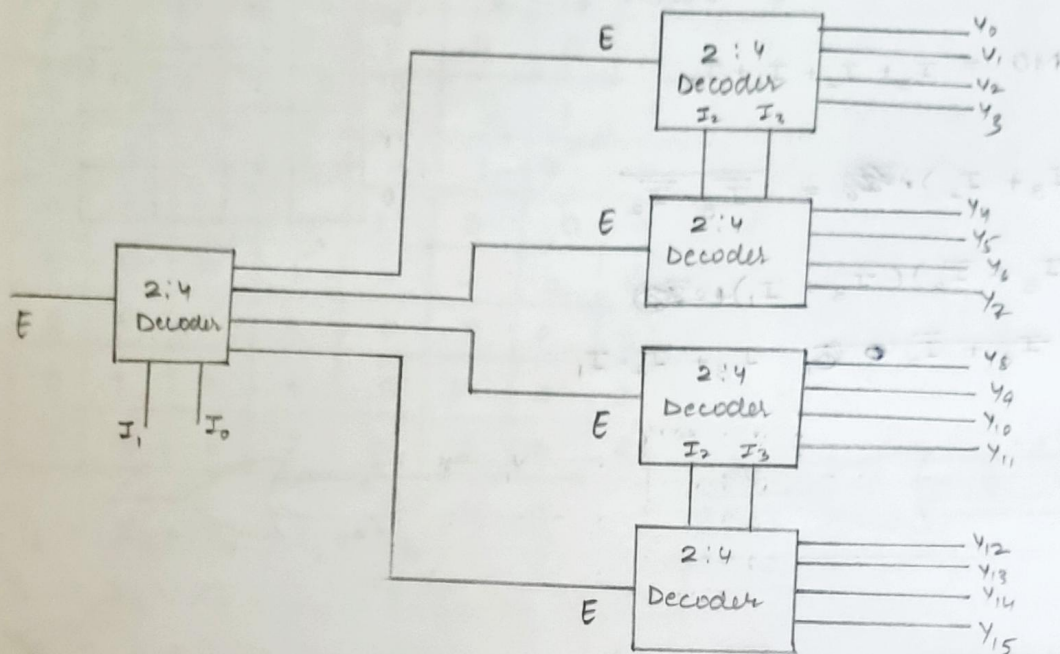
$$Y_0 = \overline{I_3} + \overline{I_2} \cdot I_1$$

$I_3 I_2$ \ $I_1 I_0$	00	01	11	10
00	X	0		
01	0	0	0	0
11				
10				



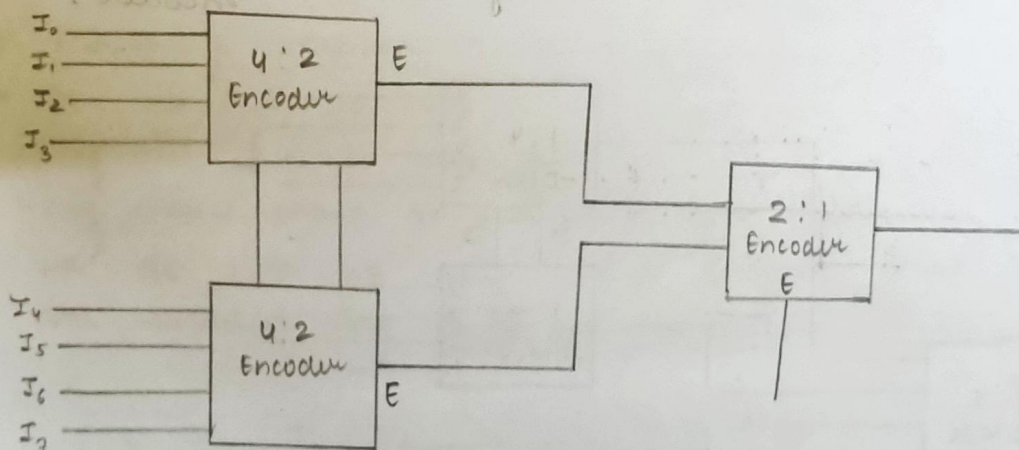
- Q Implement 1:16 D-MUX using 2:4 D-MUX.
- Q Implement 4:16 Decoder using 2:4 Decoder.
- Q Implement 8:8 Encoder using 4:2 and suitable encoder.





3. Encoder

$$\frac{8}{4} = \frac{2}{1}$$



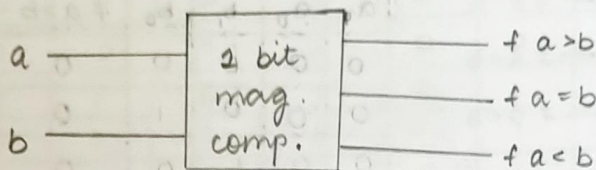
- * Data compression
- * Only identify 3 bits. (8:3)
- * Less hardware required
- * Maintains privacy

2nd Sep, 23

Magnitude Comparator

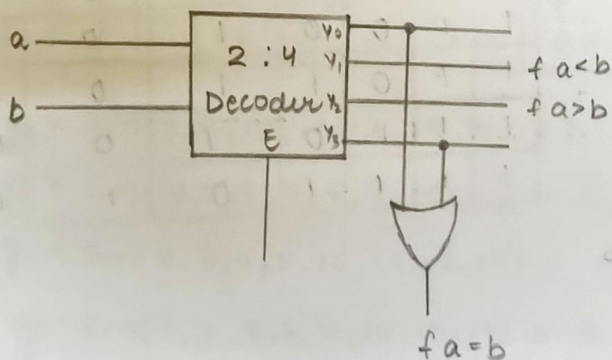
- It is a combinational circuit which compares two binary numbers in order to determine whether first number is larger, equal or smaller than second number.

1-bit magnitude comparator (Amplitude Comparator)



a	b	f a > b	f a = b	f a < b
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Q Implement 1-bit magnitude comparator using suitable decoder.

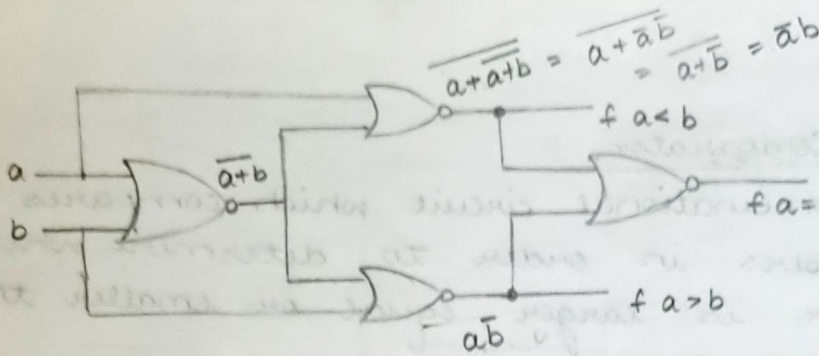


$$f a > b = a \bar{b}$$

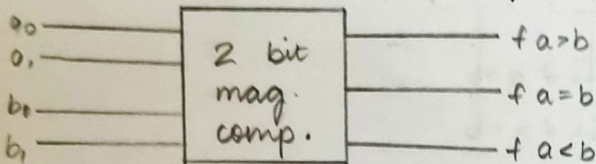
$$f a = b = a \odot b$$

$$f a < b = \bar{a} b$$

Q Implement 1-bit magnitude comparator using minimum number of NOR gate.



2-bit Magnitude Comparator



$$f(a = b) = \sum m(0, 5, 10, 15)$$

$$f(a > b) = \sum m(4, 8, 9, 12, 13, 14)$$

$$f(a < b) = \sum m(1, 2, 3, 6, 7, 11)$$

a, a_0	b_1, b_0	01	11	10
00		1	1	1
01				
11				
10				

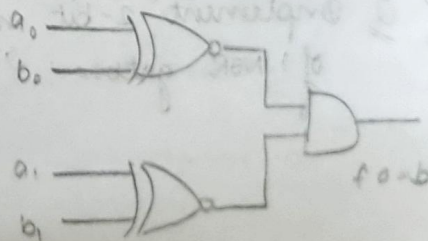
a_1	a_0	b_1	b_0	$f(a > b)$	$f(a = b)$	$f(a < b)$
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	0	1
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

$$f(a > b) = a_1 \overline{b_1} + a_0 \overline{b_1} \overline{b_0} + a_1 a_0 \overline{b_0}$$

$$f(a < b) = \overline{a_1} b_1 + \overline{a_1} \overline{a_0} b_0 + \overline{a_0} b_1 b_0$$

$$f(a = b) = (a_0 \odot b_0)(a_1 \odot b_1)$$

$$\begin{aligned}
 f(a = b) &= \overline{a_1} \overline{a_0} \overline{b_1} \overline{b_0} + \overline{a_1} a_0 \overline{b_1} b_0 + a_1 \overline{a_0} b_1 \overline{b_0} + a_1 a_0 b_1 b_0 \\
 &= \overline{a_1} \overline{b_1} (\overline{a_0} \overline{b_0} + \overline{a_0} b_0 + a_0 \overline{b_0} + a_0 b_0) \\
 &= \overline{a_1} \overline{b_1} (a_0 \odot b_0) + a_1 b_1 (a_0 \odot b_0) \\
 &= (a_0 \odot b_0)(a_1 \odot b_1)
 \end{aligned}$$



Code Converter :-

- It is a combinational circuit which converts one type of code to another type of code.

Q Design a code converter which converts 4 bit binary code into Gray code.

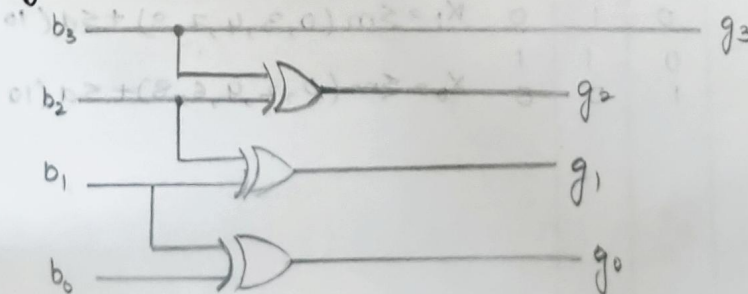
b_3	b_2	b_1	b_0	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

$$g_3 = \sum m(8, 9, 10, 11, 12, 13, 14, 15)$$

$$g_2 = \sum m(4, 5, 6, 7, 8, 9, 10, 11) = \bar{b}_3 b_2 + \bar{b}_2 b_3 = b_3 \oplus b_2$$

$$g_1 = \sum m(2, 3, 4, 5, 10, 11, 12, 13) = b_2 \oplus b_1$$

$$g_0 = \sum m(1, 2, 5, 6, 9, 10, 13, 14) = b_1 \oplus b_0$$



Q Design a code converter which converts 3 bit grey code to equivalent binary code.

0110 $\leftrightarrow$ 0101

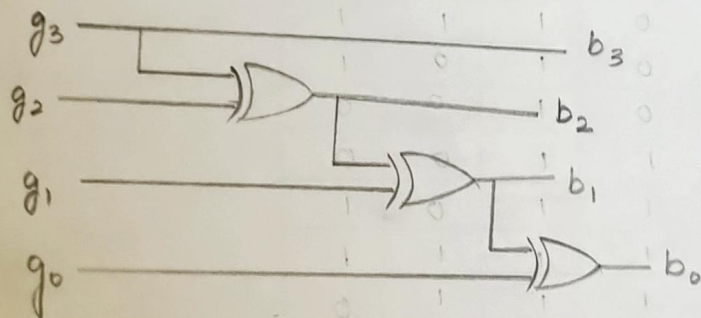
g_2	g_1	g_0	b_2	b_1	b_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	0	0
1	1	1	1	0	1

0101
 $\oplus$
 0110

$$b_2 = \sum m(4, 5, 6, 7) = g_2$$

$$b_1 = \sum m(2, 3, 4, 5) = g_2 \oplus g_1$$

$$b_0 = \sum m(1, 2, 4, 7) = g_2 \oplus g_1 \oplus g_0$$



BCD to XS-3 code converter :-

b_3	b_2	b_1	b_0	x_3	x_2	x_1	x_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

BCD $\rightarrow \sum d(10 \text{ to } 15)$

$$x_3 = \sum m(5, 6, 7, 8, 9) + \sum d(10 \text{ to } 15)$$

$$x_2 = \sum m(1, 2, 3, 4, 9) + \sum d(10 \text{ to } 15)$$

$$x_1 = \sum m(0, 3, 4, 7, 8) + \sum d(10 \text{ to } 15)$$

$$x_0 = \sum m(0, 2, 4, 6, 8) + \sum d(10 \text{ to } 15)$$

$b_3 b_2$ \ $b_1 b_0$	00	01	11	10
00				
01		1	1	1
11	d	d	d	d
10	1	1	d	d

$$X_3 = b_3 + b_2 b_0 + b_2 b_1$$

XS-3 to BCD converter.

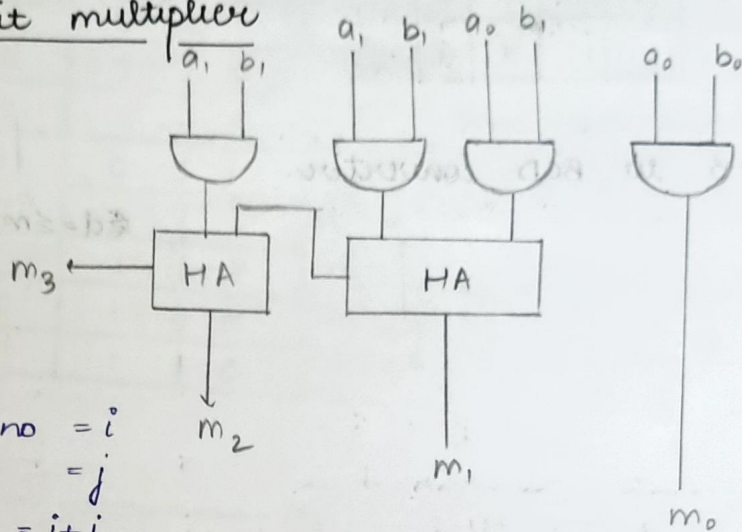
$$d = \sum m(0, 1, 2, 13, 14, 15)$$

Binary Multiplier

→ It multiplies two binary numbers of multiplier and multiplicand.

2 bit by 2 bit multiplier

$$\begin{array}{r}
 b_1 b_0 \\
 \times a_1 a_0 \\
 \hline
 c_0 = a_0 b_0 \quad a_0 b_1 \\
 a_1 b_1 \quad a_1 b_0 \\
 \hline
 m_3 \quad m_2 \quad m_1 \quad m_0 \\
 (HA) \quad (HA) \quad (HA) \quad (LSB)
 \end{array}$$



Bits in first no = i
 second = j
 Result = $i \times j$

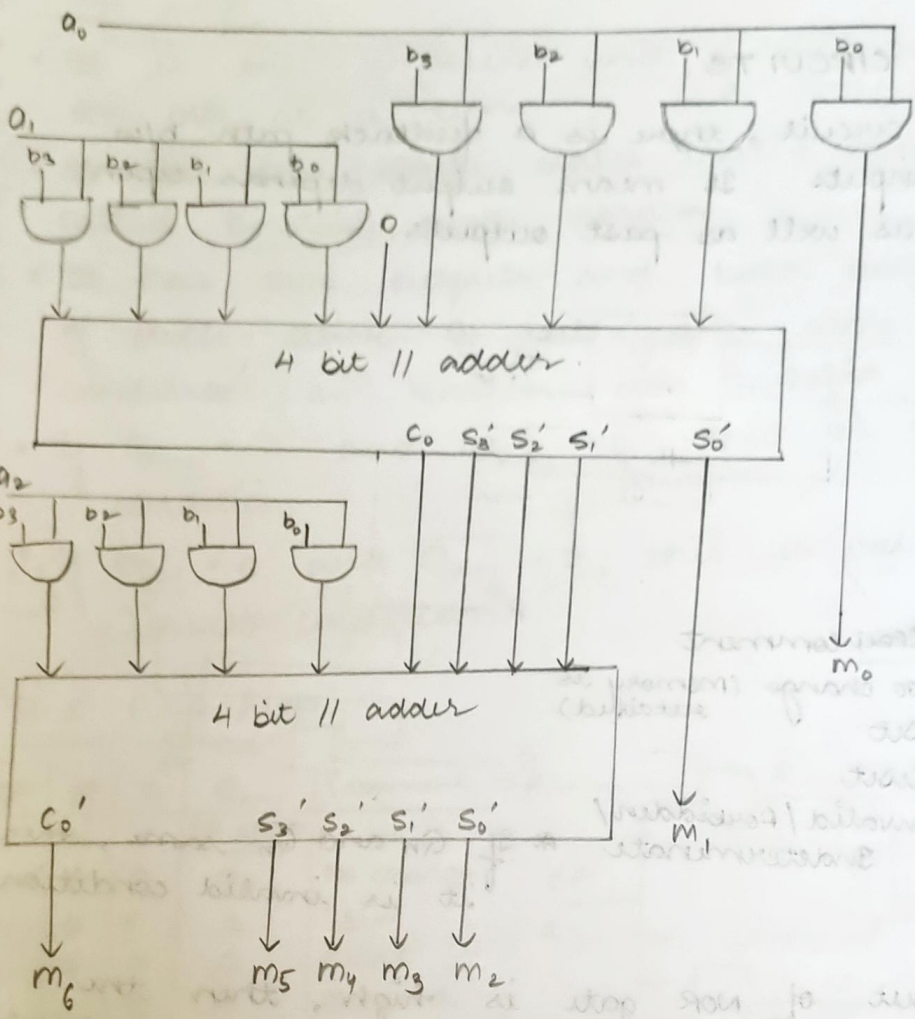
4 bit by 3 bit multiplier

$$\begin{array}{r}
 b_3 b_2 b_1 b_0 \\
 \times a_2 a_1 a_0 \\
 \hline
 a_0 b_3 \quad a_0 b_2 \quad a_0 b_1 \quad a_0 b_0 \\
 a_1 b_3 \quad a_1 b_2 \quad a_1 b_1 \quad a_1 b_0 \\
 a_2 b_3 \quad a_2 b_2 \quad a_2 b_1 \quad a_2 b_0 \\
 \hline
 \quad \quad \quad m_1 \quad m_0
 \end{array}$$

* If size of first no. is i and size of second no. is j , then total gates will be $i \times j$.

$$\text{Total} = i \times j$$

* Total parallel adder = $j - 1$.



Carry-in	$a_n + 1$	a_n	a_{n+1}
Carry-in	1	0	0
Carry-in	0	1	0
Carry-in	0	0	1
Carry-in	1	1	1

UNIT - 1

Tabular Method / Quine McClusky Method

Q Minimize following:-

$$Y = \sum m(1, 3, 9, 11, 13, 14, 15)$$

	1	1	
	1	1	1
	1	1	

	Minterms	Value
G ₁	1	0001
G ₂	3 9	0011 1001
G ₃	11 13 14	1011 1101 1110
G ₄	15	1111

Group	Minterms	Value
G ₁ G ₂	✓(1, 3) ✓(1, 9)	00-1 - 001
G ₂ G ₃	✓(3, 11) ✓(9, 11) ✓(9, 13)	- 011 10-1 1-01
G ₃ G ₄	✓(11, 15) ✓(13, 15) (14, 15)	1-11 11-1 111-

Group	Minterms	Value
G ₁ G ₂ G ₃ G ₄	(1, 3, 9, 11) (1, 9, 3, 11)	- 0 - 1 - 0 - 1
G ₂ G ₃ G ₄	(9, 11, 13, 15) (9, 13, 11, 15)	1 - - 1 1 - - 1

Prime Implicant Chart

EPI :- Single X.

Group of minterms	Minterms						
	1	3	9	11	13	14	15
(14, 15) abc ✓						X	X
(9, 11, 13, 15) ad ✓			X	X	X		X
(1, 3, 9, 11) ba ✓	X	X	X	X			

$$Y = abc + \bar{b}d + ad$$

(Sum of all EPI)

Q Minimize following function

$$Y = \sum m(9, 11, 13, 14, 15) + \sum d(1, 3)$$

	9	11	13	14	15
(14, 15) abc				X	X
(1, 3, 9, 11) $\bar{b}d$	X	X			
(9, 11, 13, 15) ad	X	X	X		X

$$Y = abc + ad$$

$$Y = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 9)$$

	1	3	7	11	15
(0, 1, 2, 3) $\bar{a}\bar{b}$	X	X			
(3, 7, 11, 15) cd		X	X	X	X
(1, 3, 9, 11) $\bar{b}d$	X	X		X	

d	1	1	d
		1	
		1	
d	1		

$$Y = cd + \bar{a}\bar{b}$$

or

$$Y = cd + \bar{b}d$$